RMIT UNIVERSITY

Report

Robot Localization with ICM20948

Dinh-Son Vu

Table of Contents

1. Pre-requisites

- Ubuntu 24.04 Noble can run on your computer (VM, UTM, or dual boot).
- ROS2 Jazzy is installed.
- This is NOT a simulation. Gazebo is no longer used. The hardware is required.
- VSCode and its extension PlatformIO (PIO) should be installed.
- First, connect the ESP32 to the computer to test the communication between ROS2 and the hardware. Later, it would be mandatory to connect the RPI to the ESP32, and ssh into the RPI.

2. Tasks

Make sure that the hardware interface is working. Normally, the joint state and the imu should already be available from the hardware interface. This lesson focuses on the integration of the hardware imu with the package robot_localization

2.1. With the esp32 connected to the computer

Tasks	Code
Clone the repo from github	git clone https://github.com/ROS2-rmitbot/lesson8_ws.git
Navigate to the workspace, build and source it	cd ~/lesson8_ws rm -rf build install log colcon build source install/setup.bash code .
Launch the bringup launch	ros2 launch rmitbot_bringup rmitbot.launch.py

Carry your robot and rotate it in the z-axis (yaw). The robot should rotate in rviz. The red arrow corresponds to the odometry from the wheel. The green arrow corresponds to the fused odometry from the robot_localization (imu+encoder).

2.2. With the esp32 connected to the RPI

This should be straightforward if the previous lesson has been performed. The only difference is the use of the imu with the robot_localization package.

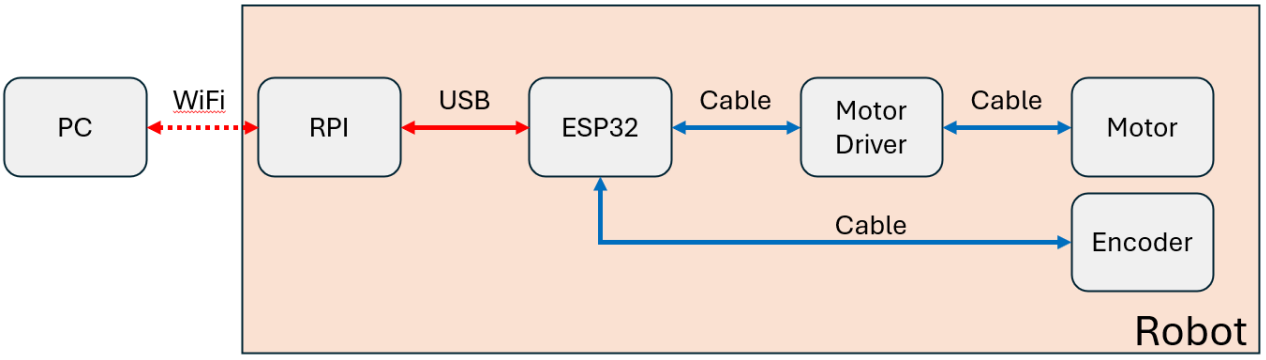

Figure 1: Robot configuration and connection

Tasks	Code
Start SSH **(make sure the RPI and PC are connected on the same WiFi)**	Open terminal → ssh username@ipaddr
On the RPI **Install serial communication library**	sudo apt install libserial-dev
On the RPI **Clone the repo from github**	git clone https://github.com/ROS2-rmitbot/lesson8_ws.git
On the RPI **Navigate to the workspace, build and source it**	cd ~/lesson8_ws rm -rf build install log colcon build source install/setup.bash
On the RPI **Launch the controller**	ros2 launch rmit_controller controller.launch.py
On the PC **Open a new terminal** **Start rviz2**	ros2 launch rmitbot_description display.launch.py
On the PC **Start teleopkeyboard**	ros2 launch rmitbot_controller teleopkeyboard.launch.py

You should be able to control the real robot from the teleoperation keyboard of the PC.

3. Explanation

There are few modifications that have been performed in the packages:

- use_sim_time parameter has been set to false in all packages
- The urdf is calling the hardware interface instead of the gazebo plugin.
- For the moment, only the hardware connected to the esp32 is tested. The integration of the Lidar and camera is covered in the next packages.

4. Packages, nodes, and topics

Packages	Nodes	Description
rmitbot_bringup		Launch the other packages and nodes
rmitbot_description	display.launch.py: rviz, rsp gazebo.launch.py: gazebo	Launch the urdf and static tf
rmitbot_controller	Controller manager: jsb spawner, controller spawner	Launch input/output of the robot
rmitbot_localization	ekf	Launch the sensor fusion between wheel encoder and imu
~~**rmitbot_mapping**~~	~~slam~~	~~Launch the lidar and slam~~
~~**rmitbot_navigation**~~	~~nav2~~	~~Launch the navigation~~
~~**rmitbot_vision**~~	~~apriltag~~	~~Include the plugin for camera. Launch the apriltag detection~~
rmitbot_firmware	Arduino firmware Hardware interface	The arduino firmware is the code flashed on the esp32. The hardware interface is the node launched with ROS2.

5. Future works

Verify that your real hardware robot is moving in the same fashion as your robot in rviz. If there is discrepancy, could you measure it?